Homework 3 - CNN Fine-Tuning

In this homework - you need to fine-tune the last layer of resnet18 to improve performance from softmax homework

of ~40% to ***acheive > 76%*** 🔥 on test set.

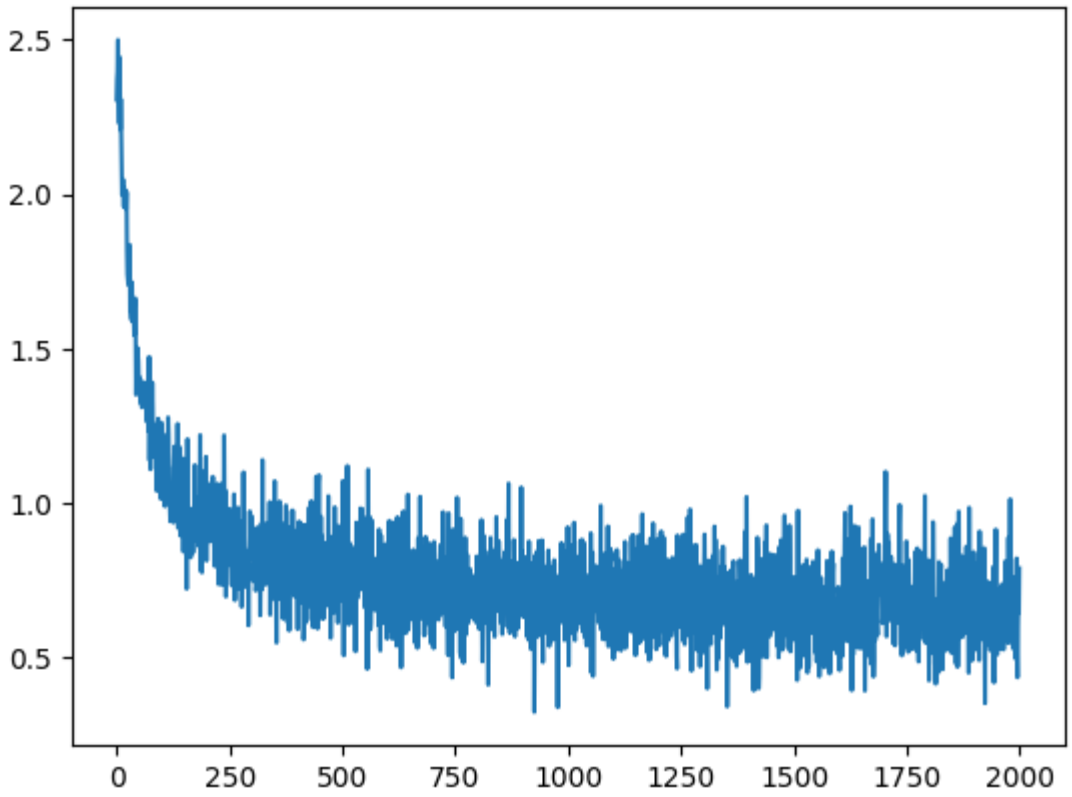
The code is divided into multiple parts. You need to complete the parts where it says

```
<<<<<<< your code here >>>>>>>>
```

I have imported all needed packages for you!

Training Loss

Your training loss should look something like this if you plot the loss after every batch.



In [1]:
```
import torch
import torch.nn as nn
import torch.optim as optim
from torchvision import datasets, transforms, models
import matplotlib.pyplot as plt
import numpy as np
from sklearn.metrics import confusion_matrix, f1_score
import seaborn as sns
from torch.utils.data import random_split
from tqdm import tqdm

'''
In this homework - you need to fine-tune the last layer of resnet18 to improve
performance from softmax homework of ~40% to acheive > 76% on test set

The code is divided into multiple parts. You need to complete the parts where
```

```
it says   <<<<<<< your code here >>>>>>>>

I have imported all needed packages for you!

Note: You need GPU for this - it takes 10-15 mins to run!

'''




# ------------------------------- Part 1 ---------------------------------------
# Check if CUDA is available and set PyTorch to use GPU or CPU
device = torch.device("cuda:0" if torch.cuda.is_available() else "cpu")

# ------------------------------------------------------------------------------

# ------------------------------- Part 2 ---------------------------------------
# Set the random seed ( dot eh same for cuda as well)

SEED = 42
torch.manual_seed(SEED)
torch.cuda.manual_seed(SEED)
generator = torch.Generator().manual_seed(SEED)
# ------------------------------------------------------------------------------

# ------------------------------- Part 3 ---------------------------------------
# Define transformations for the training data and testing data
transform = transforms.Compose([transforms.Resize(256),
                                transforms.CenterCrop(224),
                                transforms.ToTensor(),
                                transforms.Normalize([0.485, 0.456, 0.406],
                                                     [0.229, 0.224, 0.225])])



# ------------------------------- Part 4 ---------------------------------------
# Download and load the training data - split the training data into
# trainset (45000)-  valset (5000)
# you may find random_split from torch.utils.data useful!

dataset =  datasets.CIFAR10('/content/data', train=True, download=True, transform=transform)
```

```
trainset, valset =  random_split(dataset, [45000, 5000], generator=generator)
BATCH_SIZE = 32
trainloader =  torch.utils.data.DataLoader(dataset = trainset, batch_size = BATCH_SIZE)
valloader =  torch.utils.data.DataLoader(dataset = valset, batch_size = BATCH_SIZE, shuffle=True, num_workers=2)
# ---------------------------------------------------------------------------

# ------------------------------ Part 5 --------------------------------------
# Download and load the test data
testset =  datasets.CIFAR10('/content/data', train=False, download=True, transform=transform)
testloader =  torch.utils.data.DataLoader(dataset = testset, batch_size = BATCH_SIZE, shuffle=True, num_workers=2)
# ---------------------------------------------------------------------------

# ------------------------------ Part 6 --------------------------------------
# Load pre-trained model & freeze the layers
model =  models.resnet18(weights=models.ResNet18_Weights.DEFAULT)
for param in model.parameters():
    param.requires_grad = False

# for param in model.layer4.parameters():
#     param.requires_grad = True

# for param in model.fc.parameters():
#     param.requires_grad = True

# ---------------------------------------------------------------------------

# ------------------------------ Part 7 --------------------------------------

# Replace the last fully connected layer with a 10-classes layer -
NUM_OF_CLASSES = 10
num_ftrs = model.fc.in_features
model.fc = nn.Linear(num_ftrs, NUM_OF_CLASSES)

# ---------------------------------------------------------------------------

# ------------------------------ Part 8 --------------------------------------
# Move the model to the GPU if available - # <<<<<<< your code here >>>>>>>>
model =  model.to(device)
# ---------------------------------------------------------------------------

# ------------------------------ Part 9 --------------------------------------
# Define a loss function and optimizer - # <<<<<<< your code here >>>>>>>>
```

```
criterion =  nn.CrossEntropyLoss()
optimizer =  optim.SGD(model.parameters(), lr=0.0001, momentum=0.9, weight_decay=1e-3)
# ---------------------------------------------------------------------------

# ------------------------------ Part 10----------------------------------------
# Lists for loss and accuracy to plot later
train_losses = []
train_accuracy = []
val_losses = []
val_accuracy = []

# Train the model - write the training loop
'''
in the loop you need to keep track of training and validation accuracy.
Print the training & validation loss & accuracy after every epoch.

Remember when you evaluate the model on val or test set, set the model to eval
mode, and trun off graients requirement. Once you're done, set the model back to
train mode
'''
for epoch in tqdm(range(10)):  # loop over the dataset multiple times

    print(f"\nepoch - {epoch}")
    running_loss = 0.0
    correct = 0
    total = 0
    for i, data in enumerate(trainloader, 0):
        # get the inputs; data is a list of [inputs, labels]
        inputs, labels = (data[0].to(device), data[1].to(device))

        # zero the parameter gradients
        optimizer.zero_grad()


        # forward + backward + optimize
        outputs = model(inputs)
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()

        # calculate training accuracy
```

```
        _, preds = torch.max(outputs, 1)
        correct += torch.sum(preds == labels.data).cpu().item()
        total += inputs.size(0)
        running_loss += loss.item() * inputs.size(0)

        # print & accumulate training statistics (loss & accuracy) every 100 iterations
        accuracy = correct/total
        instance_loss = running_loss/total

        train_losses.append(instance_loss)
        train_accuracy.append(accuracy)

        if i % 100 == 0:
          print(f"tr-iteration - {i}")
          print(f"tr-loss - {instance_loss}")
          print(f"tr-accuracy - {accuracy}")


    # Validation at the end of the epoch
    # set mode to eval mode & tell torch no gradients are required.
    model.eval()

    # define any valuable you need to track here <<<<<
    running_loss2 = 0.0
    correct2 = 0
    total2 = 0
    with torch.no_grad():
      for i, data2 in enumerate(valloader, 0):
        inputs2, labels2 =  (data2[0].to(device), data2[1].to(device))

        outputs2 = model(inputs2)
        loss2 = criterion(outputs2, labels2)

        _, preds2 = torch.max(outputs2, 1)
        correct2 += torch.sum(preds2 == labels2.data).cpu().item()
        total2 += inputs2.size(0)
        running_loss2 += loss2.item()  * inputs2.size(0)

        accuracy2 = correct2/total2
        instance_loss2 = running_loss2/total2

        val_losses.append(instance_loss2)
```

```
        val_accuracy.append(accuracy2)

        if i % 100 == 0:
          print(f"va-iteration - {i}")
          print(f"va-loss - {instance_loss2}")
          print(f"va-accuracy - {accuracy2}")


    # set the model back to train mode
    model.train()

print('Finished Training')
# ---------------------------------------------------------------------------

# ------------------------------ Plotting -------------------------------------
# Plot training loss over time
plt.figure()
plt.plot(train_losses, label='Training Loss')
plt.plot(val_losses, label='Validation Loss')
plt.title('Training and Validation Loss')
plt.xlabel('mini-batches')
plt.ylabel('Loss')
plt.legend()

# Plot training accuracy over time
plt.figure()
plt.plot(train_accuracy, label='Training Accuracy')
plt.plot(val_accuracy, label='Validation Accuracy')
plt.title('Training and Validation Accuracy')
plt.xlabel('mini-batches')
plt.ylabel('Accuracy (%)')
plt.legend()
# ---------------------------------------------------------------------------

# ------------------------------ Part 11 --------------------------------------
# Test the model on test data and get confusion matrix
y_true = []
y_pred = []
with torch.no_grad():
    for i, data3 in enumerate(testloader, 0):
        inputs3, labels3 = (data3[0].to(device), data3[1].to(device))
```

```
        outputs3 = model(inputs3)
        _, preds3 = torch.max(outputs3, 1)
        y_true.extend(labels3.data.cpu())
        y_pred.extend(preds3.cpu())

# ---------------------------------------------------------------------------

# ------------------------------- Part 12 --------------------------------------
# Find and plot confusion matrix
conf_matrix = confusion_matrix(y_true, y_pred)
plt.figure(figsize=(10,10))
sns.heatmap(conf_matrix, annot=True, fmt='d', cmap='Blues')
plt.xlabel('Predicted Label')
plt.ylabel('True Label')
plt.title('Confusion Matrix')
plt.show()
# ---------------------------------------------------------------------------

# ------------------------------- Part 13--------------------------------------
# Print F1 score
f1 = f1_score(y_true, y_pred, average='weighted')
print('F1 Score: ', f1)
```

```
100%|██████████| 170M/170M [00:37<00:00, 4.52MB/s]
Downloading: "https://download.pytorch.org/models/resnet18-f37072fd.pth" to /root/.cache/torch/hub/checkpoints/resnet
18-f37072fd.pth
100%|██████████| 44.7M/44.7M [00:00<00:00, 136MB/s]
  0%|          | 0/10 [00:00<?, ?it/s]
```

```
epoch - 0
tr-iteration - 0
tr-loss - 2.6755170822143555
tr-accuracy - 0.09375
tr-iteration - 100
tr-loss - 2.32992152176281
tr-accuracy - 0.12283415841584158
tr-iteration - 200
tr-loss - 2.2356058483693135
tr-accuracy - 0.17412935323383086
tr-iteration - 300
tr-loss - 2.150418135414884
tr-accuracy - 0.22933970099667775
tr-iteration - 400
tr-loss - 2.077232973236693
tr-accuracy - 0.27485972568578554
tr-iteration - 500
tr-loss - 2.015362435234283
tr-accuracy - 0.312312874251497
tr-iteration - 600
tr-loss - 1.9579087985732195
tr-accuracy - 0.3456738768718802
tr-iteration - 700
tr-loss - 1.9074547051022974
tr-accuracy - 0.3719240370898716
tr-iteration - 800
tr-loss - 1.8601853457580642
tr-accuracy - 0.3971598002496879
tr-iteration - 900
tr-loss - 1.816655790369201
tr-accuracy - 0.4191870144284129
tr-iteration - 1000
tr-loss - 1.776941324804689
tr-accuracy - 0.43753121878121876
tr-iteration - 1100
tr-loss - 1.7396526568375534
tr-accuracy - 0.4545299727520436
tr-iteration - 1200
tr-loss - 1.7055270520971777
tr-accuracy - 0.4692183597002498
tr-iteration - 1300
tr-loss - 1.673259901139115
```

```
tr-accuracy - 0.48345023059185244
tr-iteration - 1400
tr-loss - 1.6438654930058927
tr-accuracy - 0.4952935403283369
va-iteration - 0
va-loss - 1.1980180740356445
va-accuracy - 0.6875
va-iteration - 100
va-loss - 1.2096601146282535
va-accuracy - 0.6642945544554455
 10%|          | 1/10 [02:08<19:20, 128.92s/it]
```

```
epoch - 1
tr-iteration - 0
tr-loss - 1.4418222904205322
tr-accuracy - 0.5
tr-iteration - 100
tr-loss - 1.2070915899654426
tr-accuracy - 0.6720297029702971
tr-iteration - 200
tr-loss - 1.2089248788890554
tr-accuracy - 0.6688432835820896
tr-iteration - 300
tr-loss - 1.1930495797597689
tr-accuracy - 0.6737956810631229
tr-iteration - 400
tr-loss - 1.1855887781057572
tr-accuracy - 0.6705112219451371
tr-iteration - 500
tr-loss - 1.1841369284365229
tr-accuracy - 0.6697230538922155
tr-iteration - 600
tr-loss - 1.1743945059879448
tr-accuracy - 0.6703410981697171
tr-iteration - 700
tr-loss - 1.1678296226407594
tr-accuracy - 0.6689550641940085
tr-iteration - 800
tr-loss - 1.1606021938699016
tr-accuracy - 0.6695926966292135
tr-iteration - 900
tr-loss - 1.151938079464581
tr-accuracy - 0.672343229744728
tr-iteration - 1000
tr-loss - 1.1440458299515845
tr-accuracy - 0.6736700799200799
tr-iteration - 1100
tr-loss - 1.1355349998599717
tr-accuracy - 0.6754654859218892
tr-iteration - 1200
tr-loss - 1.127375998614134
tr-accuracy - 0.677248126561199
tr-iteration - 1300
tr-loss - 1.1193051675848553
```

```
tr-accuracy - 0.6790449654112222
tr-iteration - 1400
tr-loss - 1.112355250149944
tr-accuracy - 0.6800499643112062
va-iteration - 0
va-loss - 1.0274349451065063
va-accuracy - 0.6875
va-iteration - 100
va-loss - 0.9732453433593901
va-accuracy - 0.7094678217821783
 20%|██        | 2/10 [04:17<17:10, 128.78s/it]
```

```
epoch - 2
tr-iteration - 0
tr-loss - 1.234557032585144
tr-accuracy - 0.59375
tr-iteration - 100
tr-loss - 0.9851804228112249
tr-accuracy - 0.7076113861386139
tr-iteration - 200
tr-loss - 0.9962901541249669
tr-accuracy - 0.7044465174129353
tr-iteration - 300
tr-loss - 0.9875663695145287
tr-accuracy - 0.709717607973422
tr-iteration - 400
tr-loss - 0.9873783649351828
tr-accuracy - 0.706359102244389
tr-iteration - 500
tr-loss - 0.993968122614596
tr-accuracy - 0.7032185628742516
tr-iteration - 600
tr-loss - 0.9898623264768159
tr-accuracy - 0.704450915141431
tr-iteration - 700
tr-loss - 0.989121924028247
tr-accuracy - 0.7022557061340942
tr-iteration - 800
tr-loss - 0.9878422149855843
tr-accuracy - 0.7017400124843945
tr-iteration - 900
tr-loss - 0.9841174785622482
tr-accuracy - 0.7035238623751388
tr-iteration - 1000
tr-loss - 0.9809400120100656
tr-accuracy - 0.7040771728271729
tr-iteration - 1100
tr-loss - 0.9768065187088691
tr-accuracy - 0.7047570390554042
tr-iteration - 1200
tr-loss - 0.9726406227142785
tr-accuracy - 0.7060002081598667
tr-iteration - 1300
tr-loss - 0.9684726135443028
```

```
tr-accuracy - 0.7067159877017679
tr-iteration - 1400
tr-loss - 0.9653850051864907
tr-accuracy - 0.7069280870806567
va-iteration - 0
va-loss - 0.6992391347885132
va-accuracy - 0.8125
va-iteration - 100
va-loss - 0.9020995975721
va-accuracy - 0.7218440594059405
 30%|▉▉▉       | 3/10 [06:25<14:57, 128.17s/it]
```

```
epoch - 3
tr-iteration - 0
tr-loss - 1.1449618339538574
tr-accuracy - 0.625
tr-iteration - 100
tr-loss - 0.8924085344418441
tr-accuracy - 0.7283415841584159
tr-iteration - 200
tr-loss - 0.905605988419471
tr-accuracy - 0.722481343283582
tr-iteration - 300
tr-loss - 0.8987149606115398
tr-accuracy - 0.7258098006644518
tr-iteration - 400
tr-loss - 0.900263199485151
tr-accuracy - 0.7218672069825436
tr-iteration - 500
tr-loss - 0.9092234364764658
tr-accuracy - 0.718999500998004
tr-iteration - 600
tr-loss - 0.9065664656547064
tr-accuracy - 0.7198939267886856
tr-iteration - 700
tr-loss - 0.9075904290448242
tr-accuracy - 0.71714514978602
tr-iteration - 800
tr-loss - 0.9082955864633663
tr-accuracy - 0.7165652309612984
tr-iteration - 900
tr-loss - 0.9061062157352545
tr-accuracy - 0.7176748057713651
tr-iteration - 1000
tr-loss - 0.9043946142320509
tr-accuracy - 0.7179070929070929
tr-iteration - 1100
tr-loss - 0.9016355486700038
tr-accuracy - 0.718352633969119
tr-iteration - 1200
tr-loss - 0.898729039171554
tr-accuracy - 0.7190362198168193
tr-iteration - 1300
tr-loss - 0.8958110438686989
```

```
tr-accuracy - 0.7194946195234435
tr-iteration - 1400
tr-loss - 0.8940550983590283
tr-accuracy - 0.7197537473233405
va-iteration - 0
va-loss - 0.5674181580543518
va-accuracy - 0.875
va-iteration - 100
va-loss - 0.8430171541058191
va-accuracy - 0.7271039603960396
 40%|████      | 4/10 [08:32<12:47, 127.99s/it]
```

```
epoch - 4
tr-iteration - 0
tr-loss - 1.0925863981246948
tr-accuracy - 0.625
tr-iteration - 100
tr-loss - 0.8398633032742113
tr-accuracy - 0.7391707920792079
tr-iteration - 200
tr-loss - 0.8536833628493162
tr-accuracy - 0.7322761194029851
tr-iteration - 300
tr-loss - 0.8475807168555022
tr-accuracy - 0.7349460132890365
tr-iteration - 400
tr-loss - 0.8497263341770505
tr-accuracy - 0.7313746882793017
tr-iteration - 500
tr-loss - 0.8596734032183588
tr-accuracy - 0.7286676646706587
tr-iteration - 600
tr-loss - 0.857551335891749
tr-accuracy - 0.7289933444259568
tr-iteration - 700
tr-loss - 0.8593802285432476
tr-accuracy - 0.7264621968616263
tr-iteration - 800
tr-loss - 0.8610509284277832
tr-accuracy - 0.7255383895131086
tr-iteration - 900
tr-loss - 0.8595611095825391
tr-accuracy - 0.726380410654828
tr-iteration - 1000
tr-loss - 0.8585015474618612
tr-accuracy - 0.7263361638361638
tr-iteration - 1100
tr-loss - 0.8563493141036592
tr-accuracy - 0.7264986376021798
tr-iteration - 1200
tr-loss - 0.8540020485553614
tr-accuracy - 0.7272845545378851
tr-iteration - 1300
tr-loss - 0.8516339061124613
```

```
tr-accuracy - 0.7278535741737125
tr-iteration - 1400
tr-loss - 0.8505122552889743
tr-accuracy - 0.7281406138472519
va-iteration - 0
va-loss - 0.6772924661636353
va-accuracy - 0.78125
va-iteration - 100
va-loss - 0.7998010941071085
va-accuracy - 0.7431930693069307
 50%|□□□□□     | 5/10 [10:38<10:36, 127.35s/it]
```

```
epoch - 5
tr-iteration - 0
tr-loss - 1.057762861251831
tr-accuracy - 0.65625
tr-iteration - 100
tr-loss - 0.8052537152082613
tr-accuracy - 0.7462871287128713
tr-iteration - 200
tr-loss - 0.8192502372893528
tr-accuracy - 0.738339552238806
tr-iteration - 300
tr-loss - 0.8136116101892288
tr-accuracy - 0.7417981727574751
tr-iteration - 400
tr-loss - 0.8160113457432412
tr-accuracy - 0.7385442643391521
tr-iteration - 500
tr-loss - 0.8264419514975861
tr-accuracy - 0.7351546906187625
tr-iteration - 600
tr-loss - 0.8245582774950938
tr-accuracy - 0.7350769550748752
tr-iteration - 700
tr-loss - 0.8268479221047417
tr-accuracy - 0.7327924393723253
tr-iteration - 800
tr-loss - 0.8290916287646014
tr-accuracy - 0.7318586142322098
tr-iteration - 900
tr-loss - 0.8279926200024164
tr-accuracy - 0.7326234739178691
tr-iteration - 1000
tr-loss - 0.8272877864308886
tr-accuracy - 0.7324862637362637
tr-iteration - 1100
tr-loss - 0.8254582928755845
tr-accuracy - 0.7325726612170754
tr-iteration - 1200
tr-loss - 0.8234101641336945
tr-accuracy - 0.7333472106577852
tr-iteration - 1300
tr-loss - 0.8213290304151707
```

```
tr-accuracy - 0.7340267102229054
tr-iteration - 1400
tr-loss - 0.820567506624068
tr-accuracy - 0.7345199857244825
va-iteration - 0
va-loss - 0.7885123491287231
va-accuracy - 0.75
va-iteration - 100
va-loss - 0.8031520347784061
va-accuracy - 0.7385519801980198
 60%|▌▌▌▌▌▌    | 6/10 [12:45<08:28, 127.01s/it]
```

```
epoch - 6
tr-iteration - 0
tr-loss - 1.0329772233963013
tr-accuracy - 0.65625
tr-iteration - 100
tr-loss - 0.7803510241579301
tr-accuracy - 0.7512376237623762
tr-iteration - 200
tr-loss - 0.7943545759317294
tr-accuracy - 0.7439365671641791
tr-iteration - 300
tr-loss - 0.7890437223586529
tr-accuracy - 0.7475083056478405
tr-iteration - 400
tr-loss - 0.7915692065570717
tr-accuracy - 0.7443110972568578
tr-iteration - 500
tr-loss - 0.8022570898551903
tr-accuracy - 0.7403318363273453
tr-iteration - 600
tr-loss - 0.8004917176610816
tr-accuracy - 0.7401206322795341
tr-iteration - 700
tr-loss - 0.8030846888005478
tr-accuracy - 0.7378298858773181
tr-iteration - 800
tr-loss - 0.8057116662220711
tr-accuracy - 0.737203495630462
tr-iteration - 900
tr-loss - 0.8048587701098371
tr-accuracy - 0.7377219755826859
tr-iteration - 1000
tr-loss - 0.8043729981759211
tr-accuracy - 0.7376998001998002
tr-iteration - 1100
tr-loss - 0.8027364732350792
tr-accuracy - 0.7377100363306085
tr-iteration - 1200
tr-loss - 0.8008676825167238
tr-accuracy - 0.7384471273938384
tr-iteration - 1300
tr-loss - 0.7989528383501303
```

```
tr-accuracy - 0.7389508070714835
tr-iteration - 1400
tr-loss - 0.7984192423059803
tr-accuracy - 0.739271056388294
va-iteration - 0
va-loss - 0.8728630542755127
va-accuracy - 0.78125
va-iteration - 100
va-loss - 0.7741924500701451
va-accuracy - 0.7435024752475248
 70%|▌▌▌▌▌▌▌   | 7/10 [14:52<06:20, 126.99s/it]
```

```
epoch - 7
tr-iteration - 0
tr-loss - 1.0145442485809326
tr-accuracy - 0.65625
tr-iteration - 100
tr-loss - 0.7613686571026793
tr-accuracy - 0.754950495049505
tr-iteration - 200
tr-loss - 0.7753051043446384
tr-accuracy - 0.7490671641791045
tr-iteration - 300
tr-loss - 0.7702502310276031
tr-accuracy - 0.7520764119601329
tr-iteration - 400
tr-loss - 0.7728455733331362
tr-accuracy - 0.7487531172069826
tr-iteration - 500
tr-loss - 0.7836752666565711
tr-accuracy - 0.7448852295409182
tr-iteration - 600
tr-loss - 0.7819722148523156
tr-accuracy - 0.7444883527454242
tr-iteration - 700
tr-loss - 0.7847842029940215
tr-accuracy - 0.7420649072753209
tr-iteration - 800
tr-loss - 0.7876885476034976
tr-accuracy - 0.7409878277153558
tr-iteration - 900
tr-loss - 0.7870030599283457
tr-accuracy - 0.7415024972253053
tr-iteration - 1000
tr-loss - 0.7866645782560735
tr-accuracy - 0.7415084915084915
tr-iteration - 1100
tr-loss - 0.7851532971609949
tr-accuracy - 0.7417404632152589
tr-iteration - 1200
tr-loss - 0.7834005918629858
tr-accuracy - 0.7424281848459617
tr-iteration - 1300
tr-loss - 0.7815889697364438
```

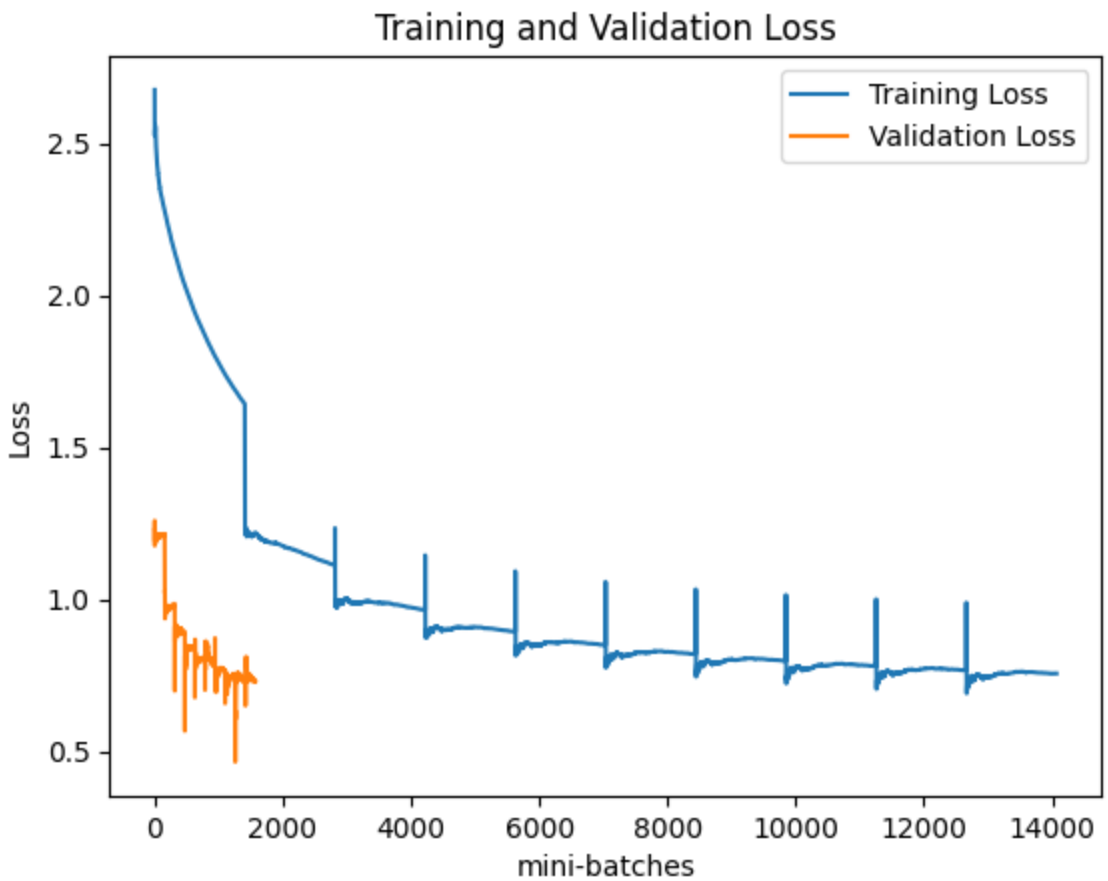
```
tr-accuracy - 0.7428900845503459
tr-iteration - 1400
tr-loss - 0.7812106420447535
tr-accuracy - 0.7431076017130621
va-iteration - 0
va-loss - 0.657935380935669
va-accuracy - 0.8125
va-iteration - 100
va-loss - 0.745905724197331
va-accuracy - 0.7521658415841584
 80%|████████  | 8/10 [16:59<04:14, 127.06s/it]
```

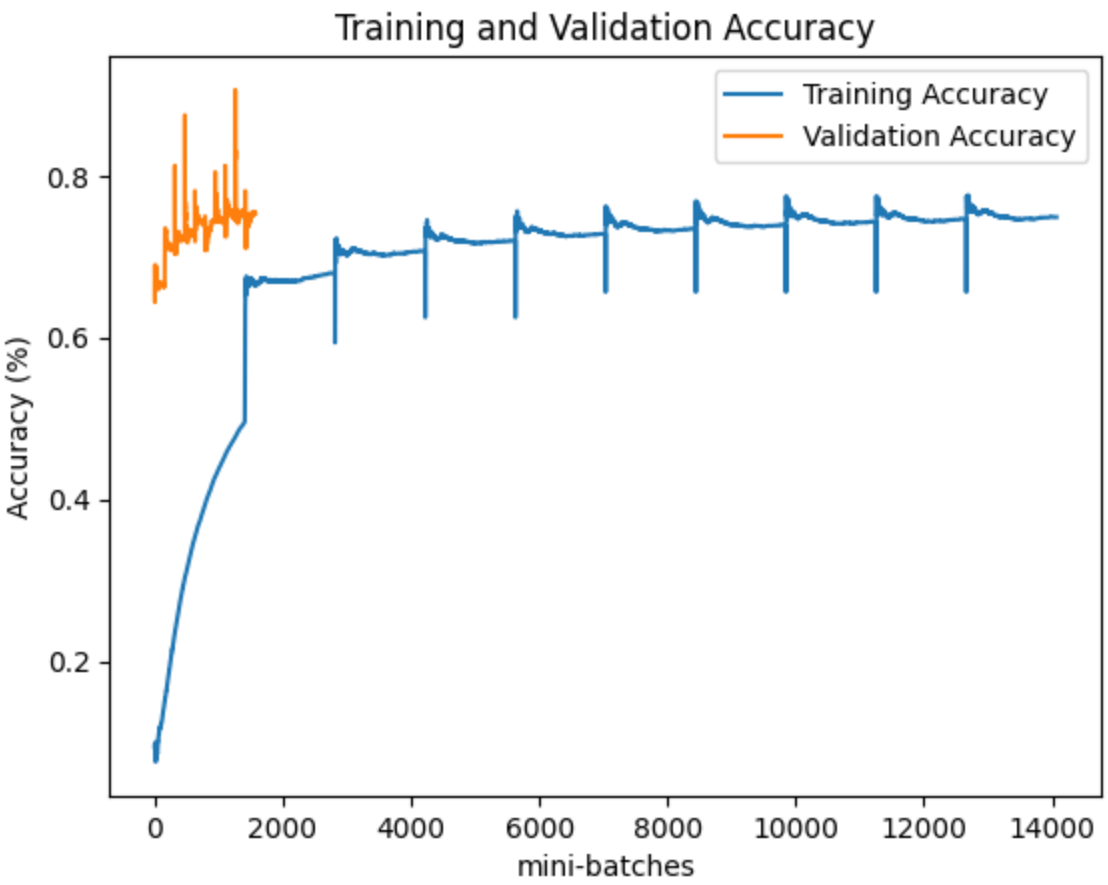
```
epoch - 8
tr-iteration - 0
tr-loss - 1.0003825426101685
tr-accuracy - 0.65625
tr-iteration - 100
tr-loss - 0.7463005922808505
tr-accuracy - 0.7574257425742574
tr-iteration - 200
tr-loss - 0.7601342153786427
tr-accuracy - 0.7501554726368159
tr-iteration - 300
tr-loss - 0.7552905003493806
tr-accuracy - 0.7534260797342193
tr-iteration - 400
tr-loss - 0.7579279541226099
tr-accuracy - 0.7502337905236908
tr-iteration - 500
tr-loss - 0.7688354049138204
tr-accuracy - 0.7467564870259481
tr-iteration - 600
tr-loss - 0.7671659409801496
tr-accuracy - 0.7466202163061564
tr-iteration - 700
tr-loss - 0.7701465252802817
tr-accuracy - 0.7445167617689016
tr-iteration - 800
tr-loss - 0.7732629331012493
tr-accuracy - 0.743484706616729
tr-iteration - 900
tr-loss - 0.7726976346559451
tr-accuracy - 0.7438956714761377
tr-iteration - 1000
tr-loss - 0.7724642518695656
tr-accuracy - 0.7441620879120879
tr-iteration - 1100
tr-loss - 0.7710394720182323
tr-accuracy - 0.7446071752951862
tr-iteration - 1200
tr-loss - 0.7693661896017171
tr-accuracy - 0.7457067027477102
tr-iteration - 1300
tr-loss - 0.7676217775680211
```

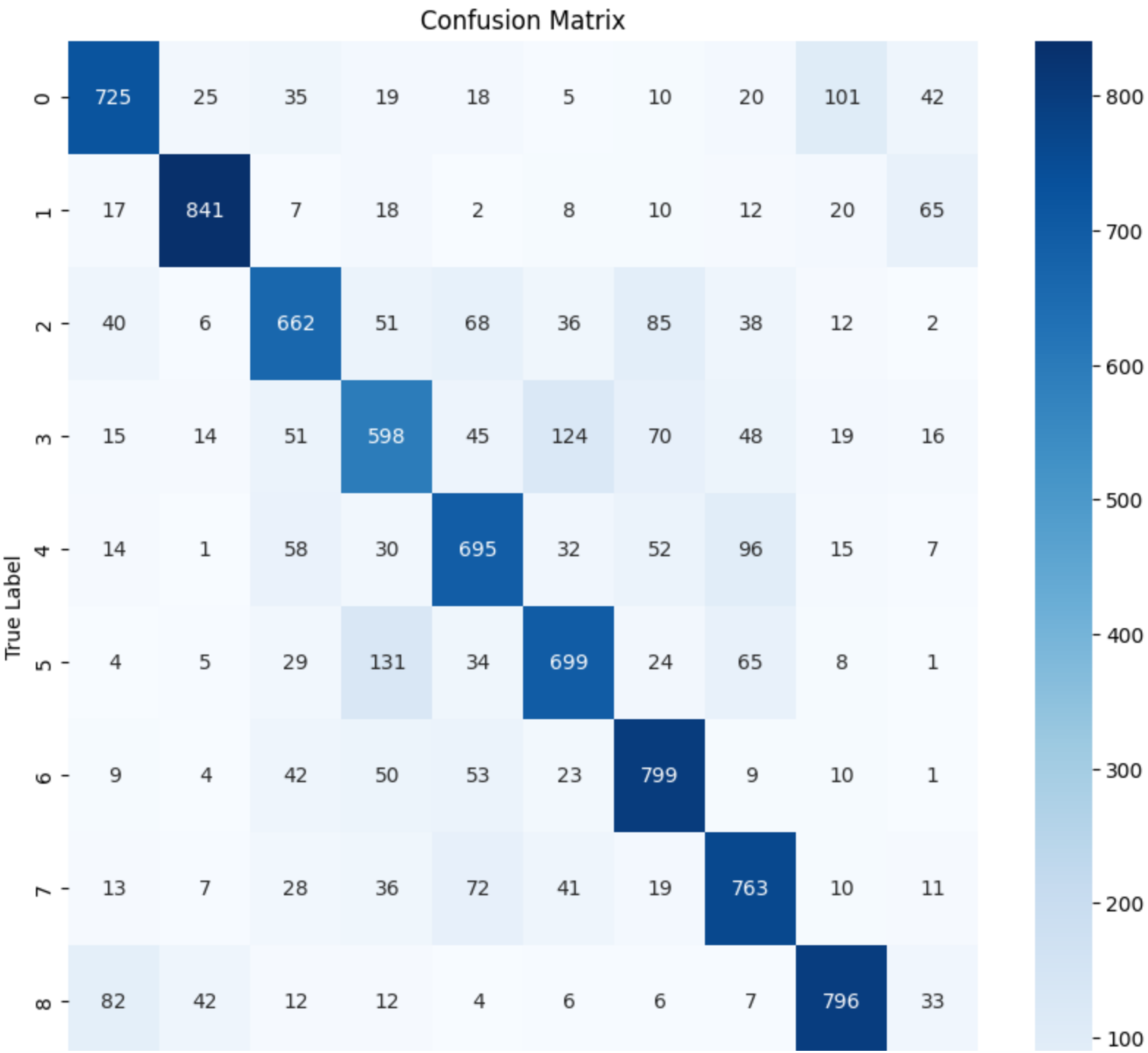
```
tr-accuracy - 0.7461808224442736
tr-iteration - 1400
tr-loss - 0.7673549635932754
tr-accuracy - 0.7462749821556032
va-iteration - 0
va-loss - 0.5692712068557739
va-accuracy - 0.84375
va-iteration - 100
va-loss - 0.7447682492213674
va-accuracy - 0.7530940594059405
 90%|██████████ | 9/10 [19:06<02:07, 127.05s/it]
```

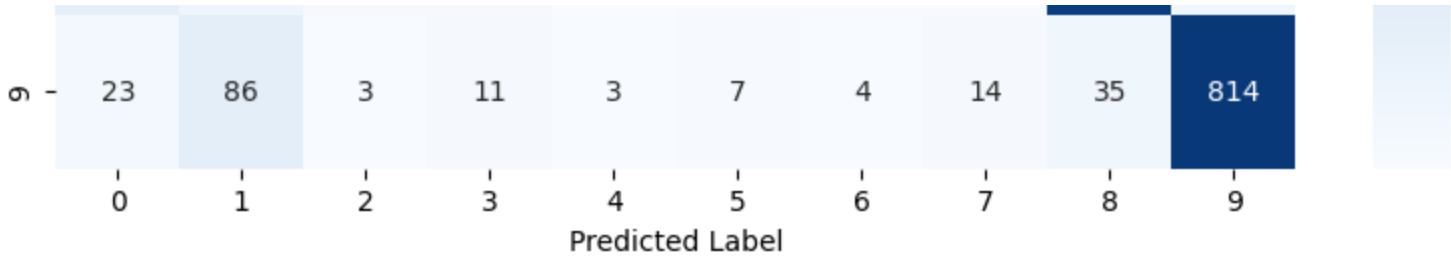
```
epoch - 9
tr-iteration - 0
tr-loss - 0.9892139434814453
tr-accuracy - 0.65625
tr-iteration - 100
tr-loss - 0.7339752608596688
tr-accuracy - 0.7620668316831684
tr-iteration - 200
tr-loss - 0.7476876914797731
tr-accuracy - 0.7541977611940298
tr-iteration - 300
tr-loss - 0.7430236400955935
tr-accuracy - 0.7570598006644518
tr-iteration - 400
tr-loss - 0.7456876048423405
tr-accuracy - 0.7537406483790524
tr-iteration - 500
tr-loss - 0.7566351791817747
tr-accuracy - 0.749812874251497
tr-iteration - 600
tr-loss - 0.7549834900509299
tr-accuracy - 0.7496360232945092
tr-iteration - 700
tr-loss - 0.7580999649540334
tr-accuracy - 0.7473252496433667
tr-iteration - 800
tr-loss - 0.7613850996065675
tr-accuracy - 0.7460596129837703
tr-iteration - 900
tr-loss - 0.7609096783710505
tr-accuracy - 0.7465663152053275
tr-iteration - 1000
tr-loss - 0.760754559959446
tr-accuracy - 0.7468156843156843
tr-iteration - 1100
tr-loss - 0.7593925676218062
tr-accuracy - 0.7471900544959128
tr-iteration - 1200
tr-loss - 0.7577759806907346
tr-accuracy - 0.7484388009991674
tr-iteration - 1300
tr-loss - 0.7560768358575483
```

```
tr-accuracy - 0.7488230207532667
tr-iteration - 1400
tr-loss - 0.7558933496262498
tr-accuracy - 0.7489293361884368
va-iteration - 0
va-loss - 0.7241671085357666
va-accuracy - 0.78125
va-iteration - 100
va-loss - 0.7430092857615782
va-accuracy - 0.7509282178217822
100%|██████████| 10/10 [21:13<00:00, 127.39s/it]
Finished Training
```




Training and Validation Accuracy
Training Accuracy
Validation Accuracy
Accuracy (%)
0.2
0.4
0.6
0.8
0
2000
4000
6000
8000
10000
12000
14000
mini-batches


Confusion Matrix
True Label
0
1
2
3
4
5
6
7
8
725 25 35 19 18 5 10 20 101 42
17 841 7 18 2 8 10 12 20 65
40 6 662 51 68 36 85 38 12 2
15 14 51 598 45 124 70 48 19 16
14 1 58 30 695 32 52 96 15 7
4 5 29 131 34 699 24 65 8 1
9 4 42 50 53 23 799 9 10 1
13 7 28 36 72 41 19 763 10 11
82 42 12 12 4 6 6 7 796 33
800
700
600
500
400
300
200
100




```
F1 Score:  0.7384321296962666
```